

RAG Issue Fix: Cheapest Delivery Option

Project: eCOM-AI | Area: AI chat retrieval and answer ranking

Short version: The app was retrieving the right delivery sources, but it ranked them by AI similarity score only. For a cheapest-price question, we fixed this by reordering retrieved matches using the real numeric price metadata.

1. The User Question

The problematic question was:

Which delivery option is cheapest?

The expected answer should be FREE delivery , because its cost is \$0.00 .

2. What Was Happening Before

The RAG flow was already doing useful work:

- It detected the question as delivery-related.
- It retrieved delivery documents.
- It included FREE delivery in the sources.

But the top result was wrong:

Rank Before Fix	Delivery Option	Price	Why It Appeared There
1	UPS3 delivery	\$2.55	Higher semantic/vector score for the word "cheap"
2	FREE delivery	\$0.00	Correct cheapest option, but lower semantic/vector score
3	UPS1 delivery	\$10.00	Relevant delivery option, but not cheap

3. Root Cause

The app was treating "cheapest" like a language similarity problem. That is only half right.

For this question, the app must combine:

- **Retrieval:** Find delivery options relevant to the question.
- **Structured ranking:** Sort those relevant options by actual numeric price.

The retrieval layer found the right documents, but the answer layer used the first semantic match as the top answer.

4. Remedy Taken

We added a structured ordering step in `Infrastructure/Services/AiChatService.cs` .

Instead of immediately taking the first 3 vector results, the service now:

- Retrieves candidate matches.
- Checks whether the question asks for `cheapest` , `lowest price` , or `least expensive` .
- Sorts those matches by real `metadata["price"]` .
- Uses AI score only as a tie-breaker when prices are equal.

5. Code That Fixes It

```
return matches
    .OrderBy(match => TryGetMetadataDouble(match.Chunk, "price", out var price)
        ? price
        : double.MaxValue)
    .ThenByDescending(match => match.Score)
    .ThenBy(match => match.Chunk.Title)
    .ToList();
```

How This Works

Step	Meaning
<code>OrderBy(price)</code>	Lowest price comes first, so <code>\$0.00</code> beats <code>\$2.55</code> .
<code>double.MaxValue</code>	If a match has no price, push it to the end instead of letting it win.
<code>ThenByDescending(score)</code>	If two options have the same price, prefer the stronger AI retrieval match.
<code>ThenBy(title)</code>	If still tied, sort alphabetically so the output is stable.

6. Example After Fix

Input question:

```
Which delivery option is cheapest?
```

Result after the fix:

```
I found these relevant delivery options: FREE delivery, UPS3 delivery, UPS2 delivery.
The top match is FREE delivery. Delivery option: FREE. Estimated delivery time:
1-2 Weeks. Cost: $0.00. Details: Free! You get what you pay for
```

Rank After Fix	Delivery Option	Price	Reason
1	FREE delivery	\$0.00	Lowest numeric price

Rank After Fix	Delivery Option	Price	Reason
2	UPS3 delivery	\$2.55	Next cheapest retrieved option
3	UPS2 delivery	\$5.00	Next cheapest retrieved option

7. Verification

- **Build passed** using `dotnet build API/API.csproj`.
- **Live API retest passed** for `Which delivery option is cheapest?`.
- **Unsupported question behavior still works** for `Do you offer cash on delivery?`.
- **Product cheapest behavior still works** for `Which hat is cheapest?`.

8. Lesson For RAG

RAG should not use vector similarity for everything. Vector retrieval is good at finding relevant text, but business questions often need structured rules after retrieval.

For this app:

- Use AI/vector search to find relevant products, delivery methods, and policies.
- Use structured fields like `price`, `deliveryTime`, `brand`, and `type` for exact filtering and sorting.
- Use an LLM later only to write a natural final answer from the verified sources.

9. Remaining Nice-To-Have

Still not full LLM generation: The app still uses template wording like "I found these relevant results." The retrieval and ranking are now better, but an LLM can later improve answer wording while staying grounded in the retrieved sources.